

vlm_worker.py 파일 정리

전체 코드

```
# vlm_worker.py

import threading
import queue
import torch
import gc

from vlm_person_analyzer_Qwen_test import PersonAnalyzer

class VLMWorker:
    def __init__(self, state_manager):
        self.state_manager = state_manager
        self.task_queue = queue.Queue()
        self.running = False
        self.thread = None
        self.analyzer = None

    def start(self):
        if self.thread is not None and self.thread.is_alive():
            print("VLM Worker 이미 실행 중")
            return

        self.running = True
        self.thread = threading.Thread(target=self._run, daemon=True)
        self.thread.start()

    def add_task(self, person_id, crop_path):
        if not self.running:
            return

        self.task_queue.put((person_id, crop_path))

    def _run(self):
        try:
            print("VLM 모델 로딩 중...")
            self.analyzer = PersonAnalyzer()
            print("VLM 모델 로딩 완료")
        except Exception as e:
            print(f"VLM 모델 로딩 실패: {e}")
```

```

        self.running = False
        return

    while self.running:
        try:
            person_id, crop_path = self.task_queue.get(timeout=1)
        except queue.Empty:
            continue

        try:
            print(f"ID {person_id} VLM 분석 시작: {crop_path}")

            result = self.analyzer.analyze(crop_path)

            self.state_manager.mark_vlm_done(person_id, result)

            print(f"ID {person_id} VLM 분석 결과:")
            print(result)

        except Exception as e:
            print(f"ID {person_id} VLM 분석 실패: {e}")

        finally:
            self.task_queue.task_done()

    self.cleanup()

    def stop(self):
        self.running = False

        if self.thread is not None and self.thread.is_alive():
            self.thread.join(timeout=10)

        self.cleanup()

    def cleanup(self):
        if self.analyzer is not None:
            try:
                del self.analyzer
            except Exception:
                pass

        self.analyzer = None

    gc.collect()

```

```
if torch.cuda.is_available():  
    torch.cuda.empty_cache()
```

클래스 역할

VLM분석을 메인 영상 처리와 분리해서 백그라운드 스레드에서 실행하는 작업 관리자 역할
메인 CCTV 루프가 멈추지 않도록 분석 요청을 큐에 저장하고, 별도 스레드가 순서대로 crop 이미지를 분석

함수별 역할

__init__()

- 입력값
 - state_manager - 사람 상태 관리 객체
- 프로세스
 - 상태 관리자 저장
 - 작업 큐 생성
 - Worker 실행 상태 변수 생성
 - 스레드 변수 초기화
 - VLM 분석기 변수 초기화
- 출력값 없음

start()

- 입력값 없음
- 프로세스
 - 이미 Worker 스레드 실행중인지 확인
 - 실행 상태 True 변경
 - 백그라운드 스레드 생성
 - _run() 함수 실행 연결
 - 스레드 시작
- 출력값 없음

add_task()

- 입력값
 - person_id - 분석 대상 person_id
 - crop_path - crop 이미지 경로
- 프로세스
 - Worker 실행 여부 확인

- 작업 큐에 (person_id, crop_path) 작업 추가
- 출력값 없음

`_run()` - 클래스 내부 함수

- 입력값 없음
- 프로세스
 - VLM 모델 로딩 - `PersonAnalyzer()` 생성
 - 실행중인 동안 반복 수행
 - 큐에서 작업 가져오기
 - crop 이미지 VLM 분석 수행
 - 분석 결과 `state_manager`에 저장
 - 작업 완료 처리
 - Worker 종료시 `cleanup()` 호출
- 출력값 없음

`stop()`

- 입력값 없음
- 프로세스
 - 실행 상태 `False` 변경
 - 스레드 종료 대기
 - `cleanup()` 호출하여 메모리 정리
- 출력값 없음

`cleanup()`

- 입력값 없음
- 프로세스
 - VLM 모델 객체 제거
 - Python 메모리 정리
 - `gc.collect()`
 - CUDA GPU 메모리 정리
 - `torch.cuda.empty_cache()`
- 출력값 없음

지피티가 짜준 핵심 정리

함수	입력값	주요 처리	출력값
<code>__init__()</code>	<code>state_manager</code>	Worker 초기 설정	없음

함수	입력값	주요 처리	출력값
start()	없음	Worker 스레드 시작	없음
add_task()	person_id, crop_path	분석 작업 Queue 추가	없음
_run()	없음	Queue 작업 순차 VLM 분석	없음
stop()	없음	Worker 종료	없음
cleanup()	없음	GPU/메모리 정리	없음